

Time Complexity

DAY 32
11/08/2025

How long the code or the Algorithm take time to execute is measured as Time Complexity

The time complexity can be calculated mathematically without using any device.

How Time Complexity Is Measured

1. Identify the Basic Operations
2. Count the Basic Operations
3. Express the Count as a function
4. Simplify the function
5. Use Big O Notation.

Basic Operations $\rightarrow$ Determine the most significant operations in the algorithm. These are operations that most affect the algorithm's runtime. (Ex: Comparisons, assignments, arithmetic operation)

Count the Basic Operations $\rightarrow$ Count how many times these basic operations are executed as a function of the input size n . This involves analyzing loops, recursive calls, and any other construct that affects the runtime.

Express the count as a function $\rightarrow$

Express the total number of operations as a mathematical function of n , where n is the size of the input.

Simplify the function

Simplify the function only by keeping the dominant terms. Which is the term that grows the fastest as " n " increases. Discard constants

and less significant terms because they have negligible effect on the growth rate for large "N"

Use Big O notation

Express the simplified function using Big O notation to describe the time complexity.

NOTE $\rightarrow$ Big O Notations provide an upper bound on the growth rate focusing on the dominant term.

Some Notations

constant $O(1)$

Taking array index element access as Example

Numbers	Index	0	1	2	3	4	5
	Value	1	2	3	7	8	9

$\text{Numbers}[3] = 7$

constant time

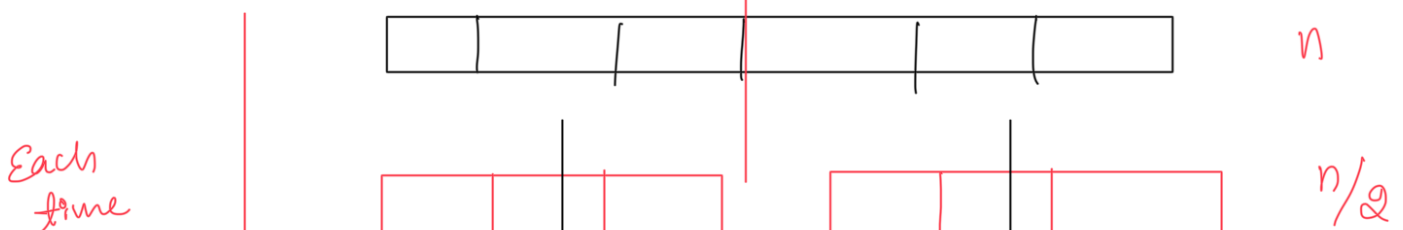
array element accessing is done by $O(1)$ constant time

its Not dependent of Array size

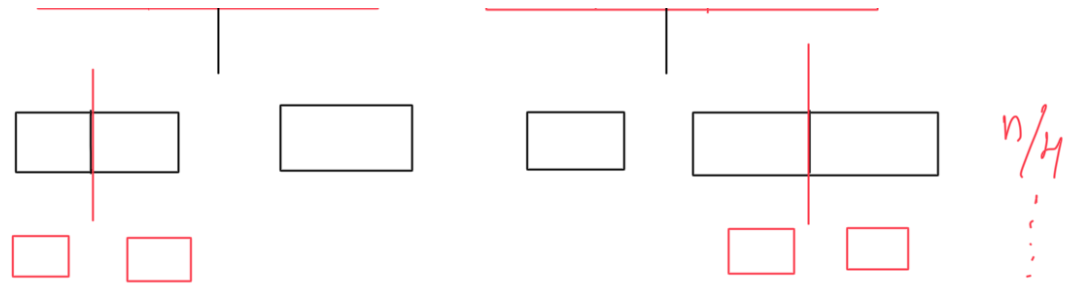
The 3rd index element is directly accessed without any delay

Logarithmic Search $O(\log n)$

Binary Search as Example. (sorted Array)



Array is divided



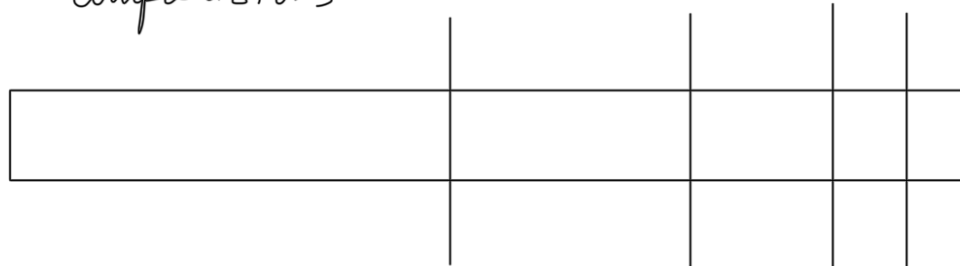
Suppose $N = 100$ In worst case we have to do 10 Comparisons

$$\frac{100}{2} + \frac{100}{4} + \frac{100}{8} + \frac{100}{16} + \frac{100}{32} + \frac{100}{64}$$

$$50 + 25 + 12 + 6 + 3 + 2 + 1$$

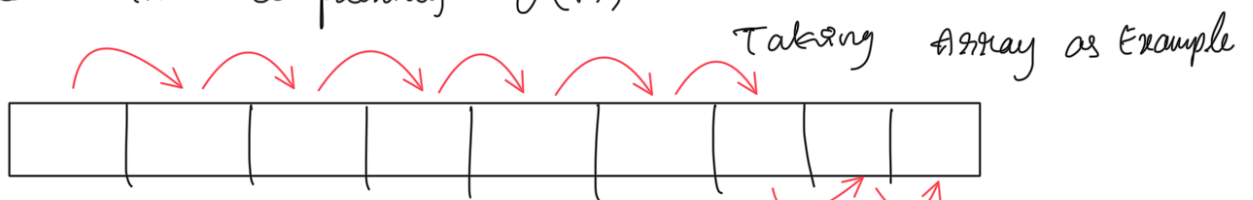
$$50 \quad 25 \quad 12 \quad 6 \quad 3 \quad 2 \quad 1$$

$\log_2 100$ Comparisons



Search operation decreases each time

Linear Time Complexity $O(n)$



going on each index and Searching for element in unsorted Array.

In worst case we have to traverse the whole Array.

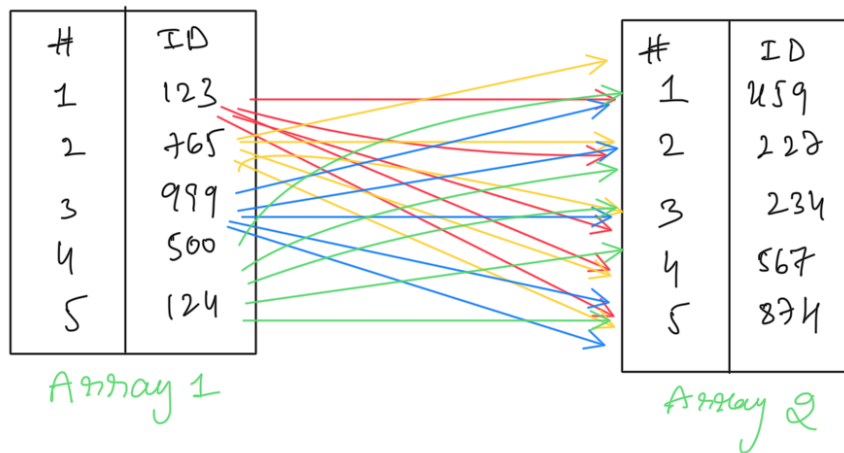
which means n comparisons max.

$O(n)$ where n is size of array.

If array is unsorted then there is no way of optimised search we have to traverse through array one by one linearly until we get elements.

Quadratic Time Complexity

finding common elements Btw Two arrays.



To find common elements in the Array 1 & Array 2 we have to take each and every element and we have to search linearly.

It's like performing linear search for each element in the array.

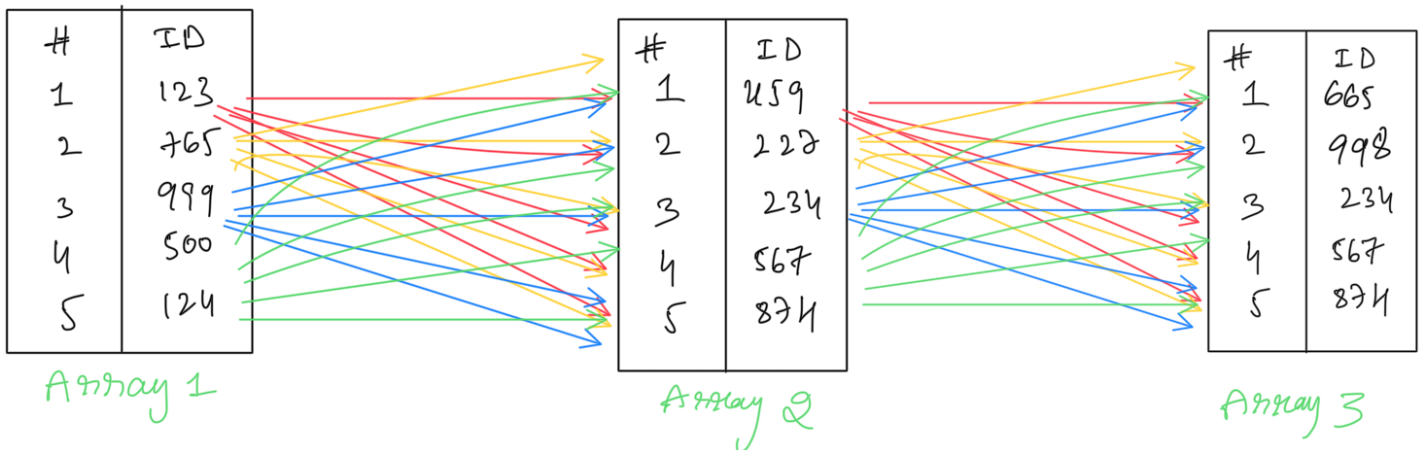
$$n \times n = O(n^2)$$

Cubic time complexity

taking 3 arrays and finding common elements

it performing n^2 comparisons n times

$$n^2 \times n = n^3$$

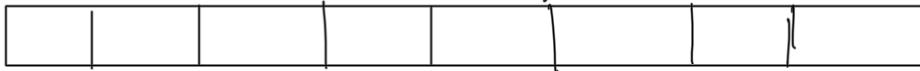


Exponential time complexity (2^n)

Taking Array subset as Example.

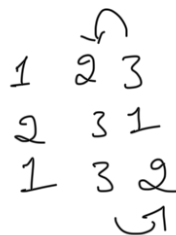
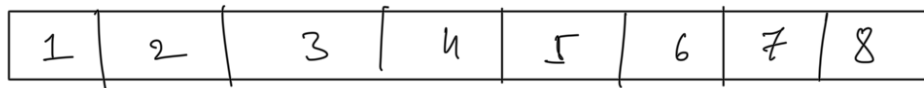


printing all possible subsets of the array.

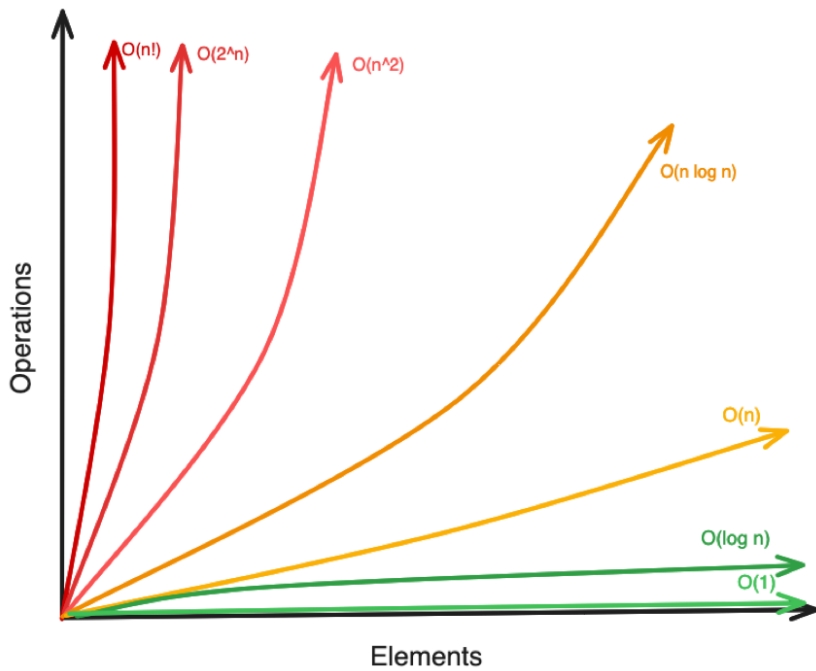


Factorial $O(n!)$

Generate all possible Seating Arrangements of n peoples in a class as Example.



Which time Complexity is Best ?



$$O(1) < O(\log n)$$

$$O(\log n) < O(n) < O(n \log n)$$

$$O(n \log n) < O(n^2)$$

$$O(n^2) < O(2^n)$$

$$O(2^n) < O(n!)$$

The Algorithm with the least Time Complexity is most efficient.
